



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра информационной безопасности

Суперкомпьютерное решение задачи разностной схемы для уравнения Пуассона задачи Дирихле

ОТЧЕТ ПО ВЫПОЛНЕНИЮ ЗАДАНИЙ ПО КУРСУ
«СУПЕРКОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ И ТЕХНОЛОГИИ»

Студент дневного отделения магистратуры

Лю Хайлинь

Преподаватель:

Доцент, Попова Н.Н.

Москва, 2025

Содержание

1	Введение	3
1.1	Актуальность	3
1.2	Постановка задачи	3
2	Математическая постановка и численные методы	5
2.1	Основное уравнение	5
2.2	Метод фиктивных областей	5
2.3	Метод конечных разностей	6
2.4	Метод сопряжённых градиентов с предобуславливанием	7
3	Программная реализация и результатов выполнения	9
3.1	Последовательная	9
3.2	Распараллеливание с OpenMP	9
3.3	Распараллеливание с MPI	13
3.4	Распараллеливание с MPI + OpenMP	15
3.5	Распараллеливание с MPI + CUDA	17
3.6	Анализ производительности	22
4	Заключение	26

1 Введение

1.1 Актуальность

Решение уравнения Пуассона имеет фундаментальное значение в математическом моделировании процессов теплопередачи, электростатического потенциала, диффузии и гидродинамики. Для большинства реальных задач, где область имеет сложную геометрию или неоднородные граничные условия, аналитические решения отсутствуют, поэтому требуется численное моделирование.

Современные вычислительные методы и технологии параллельных вычислений позволяют эффективно решать краевые задачи больших размерностей. Применение параллельных моделей, таких как OpenMP и MPI, обеспечивает значительное ускорение вычислений на многоядерных и многопроцессорных архитектурах.

Дополнительный рост производительности достигается за счёт использования графических ускорителей (GPU). Архитектура CUDA предоставляет программную модель с тысячами параллельных потоков и значительно более высокой пропускной способностью памяти по сравнению с традиционными CPU. Благодаря этому, GPU особенно эффективны при расчётах, содержащих регулярные сеточные структуры и большие объёмы однотипных операций — таких как вычисление дискретного лапласиана и обновление векторов в методе сопряжённых градиентов.

Таким образом, исследование производительности различных параллельных подходов — OpenMP, MPI и MPI+CUDA — представляет собой актуальную задачу суперкомпьютерных технологий. Сравнительный анализ позволяет оценить влияние архитектуры вычислительной системы на эффективность решения уравнения Пуассона и определить оптимальную стратегию распараллеливания в зависимости от доступных вычислительных ресурсов.

1.2 Постановка задачи

Цели работы

- Разработать и реализовать численный метод решения уравнения Пуассона в трапециевидной области с помощью метода конечных разностей.

- Исследовать эффективность параллельных реализаций на основе OpenMP, MPI и гибридной модели MPI+OpenMP.
- Реализовать версию программы на GPU с использованием CUDA и провести анализ её производительности.
- Провести сравнительный анализ ускорений и масштабируемости разработанных реализаций.

Задачи работы

- Построить разностную схему для уравнения Пуассона с использованием метода фиктивных областей.
- Реализовать итерационный метод сопряжённых градиентов для решения полученной системы линейных уравнений.
- Реализовать параллельные версии программы с применением OpenMP, MPI, гибридной схемы MPI+OpenMP и модели MPI+CUDA.
- Провести вычислительные эксперименты для различных размеров сеток и числа процессов/устройств.
- Проанализировать влияние параллелизма и архитектуры вычислителей на скорость сходимости, производительность и эффективность различных реализаций.

2 Математическая постановка и численные методы

2.1 Основное уравнение

Рассматривается двумерное уравнение Пуассона задачи Дирихле:

$$-\Delta u = f(x, y), \quad (x, y) \in D, \quad (2.1)$$

где Δ — оператор Лапласа:

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}. \quad (2.2)$$

Для заданной области D с границей ∂D формулируются граничные условия Дирихле:

$$u(x, y) = 0, \quad (x, y) \in \partial D. \quad (2.3)$$

Правая часть уравнения полагается постоянной:

$$f(x, y) = 1. \quad (2.4)$$

Область D представляет собой трапецию с вершинами

$$A(0, 0), \quad B(3, 0), \quad C(2, 3), \quad D(0, 3),$$

вложенную в прямоугольник $\Pi = [0, 3] \times [0, 3]$.

2.2 Метод фиктивных областей

Для нахождения приближённого решения задачи (2.1)–(2.3) используется метод конечных разностей. Поскольку рассматриваемая область D имеет сложную форму (трапецию), прямое построение сетки, совпадающей с её границей, является затруднительным. Для этого применяется **метод фиктивных областей**, позволяющий рассматривать область D как часть прямоугольной области Π , в которой удобно строить регулярную сетку.

Пусть область D принадлежит прямоугольнику Π , заданному как

$$\Pi = \{(x, y) : A_1 < x < B_1, A_2 < y < B_2\},$$

а $\hat{D} = \Pi \setminus D$ — фиктивная область (область вне D , но внутри Π).

Тогда исходное уравнение (2.1) переписывается в обобщённом виде:

$$-\frac{\partial}{\partial x} \left(k(x, y) \frac{\partial v}{\partial x} \right) - \frac{\partial}{\partial y} \left(k(x, y) \frac{\partial v}{\partial y} \right) = F(x, y), \quad (x, y) \in \Pi, \quad (2.5)$$

с граничным условием:

$$v(x, y) = 0, \quad (x, y) \in \Gamma = \partial\Pi, \quad (2.6)$$

где $k(x, y)$ — коэффициент теплопроводности, а $F(x, y)$ — обобщённая правая часть, определяемые следующим образом:

$$k(x, y) = \begin{cases} 1, & (x, y) \in D, \\ \frac{1}{\varepsilon}, & (x, y) \in \widehat{D}, \end{cases} \quad F(x, y) = \begin{cases} f(x, y), & (x, y) \in D, \\ 0, & (x, y) \in \widehat{D}, \end{cases} \quad (2.7)$$

где ε — малый параметр, обеспечивающий плавное выключение уравнения в фиктивной области.

2.3 Метод конечных разностей

Для построения численного решения уравнения (2.5) используется **метод конечных разностей**, который заключается в аппроксимации производных разностями на равномерной прямоугольной сетке.

В прямоугольной области $\Pi = [A_1, B_1] \times [A_2, B_2]$ задаётся равномерная сетка

$$x_i = A_1 + ih_x, \quad i = 0, 1, \dots, M, \quad y_j = A_2 + jh_y, \quad j = 0, 1, \dots, N, \quad (2.8)$$

где шаги сетки определяются как

$$h_x = \frac{B_1 - A_1}{M}, \quad h_y = \frac{B_2 - A_2}{N}.$$

В каждой внутренней узловой точке (x_i, y_j) производные в уравнении (2.5) аппроксимируются центральными разностями. Получаем разностную схему в виде:

$$\begin{aligned} & -\frac{1}{h_x^2} \left(k_{i+\frac{1}{2},j} (u_{i+1,j} - u_{i,j}) - k_{i-\frac{1}{2},j} (u_{i,j} - u_{i-1,j}) \right) \\ & -\frac{1}{h_y^2} \left(k_{i,j+\frac{1}{2}} (u_{i,j+1} - u_{i,j}) - k_{i,j-\frac{1}{2}} (u_{i,j} - u_{i,j-1}) \right) = F_{i,j}. \end{aligned} \quad (2.9)$$

где $u_{i,j} \approx u(x_i, y_j)$, а коэффициенты $k_{i\pm\frac{1}{2},j}$ и $k_{i,j\pm\frac{1}{2}}$ вычисляются как средние значения $k(x, y)$ между соседними узлами:

$$k_{i+\frac{1}{2},j} = \frac{1}{2} (k_{i,j} + k_{i+1,j}), \quad k_{i,j+\frac{1}{2}} = \frac{1}{2} (k_{i,j} + k_{i,j+1}). \quad (2.10)$$

Правая часть $F_{i,j}$ определяется как значение функции $F(x, y)$ в узле (x_i, y_j) :

$$F_{i,j} = F(x_i, y_j). \quad (2.11)$$

На границе $\Gamma = \partial\Pi$ реализуются граничные условия Дирихле:

$$u_{i,j} = 0, \quad (x_i, y_j) \in \Gamma. \quad (2.12)$$

Таким образом, для всех внутренних точек сетки $(i = 1, \dots, M - 1, j = 1, \dots, N - 1)$ уравнение (2.9) задаёт систему линейных алгебраических уравнений относительно неизвестных $u_{i,j}$:

$$Au = F,$$

где A — дискретный оператор Лапласа, модифицированный коэффициентами $k(x, y)$, а F — вектор правых частей.

2.4 Метод сопряжённых градиентов с предобуславливанием

После дискретизации получаем СЛАУ

$$Au = F, \quad A = A^\top \succ 0, \quad (2.13)$$

эквивалентную задаче минимизации квадратичного функционала

$$J(u) = \frac{1}{2} (Au, u) - (F, u). \quad (2.14)$$

Градиент функционала равен

$$\nabla J(u) = Au - F = -(F - Au) = -r, \quad r := F - Au, \quad (2.15)$$

поэтому движение в направлении r (или его предобусловленного варианта $z = D^{-1}r$) уменьшает J .

Для ускорения сходимости применяется диагональный предобуславливатель $D \approx A$:

$$(Dw)_{i,j} = \left(\frac{k_{i+1,j} + k_{i,j}}{h_x^2} + \frac{k_{i,j+1} + k_{i,j}}{h_y^2} \right) w_{i,j}, \quad 1 \leq i \leq M - 1, \quad 1 \leq j \leq N - 1, \quad (2.16)$$

что даёт простое обращение $z = D^{-1}r$ покомпонентно.

Строить направления $p^{(k)}$, попарно A -сопряжённые,

$$(Ap^{(k)}, p^{(\ell)}) = 0, \quad k \neq \ell, \quad (2.17)$$

и искать $u^{(k+1)} = u^{(k)} + \alpha_k p^{(k)}$ с оптимальным шагом

$$\alpha_k = \frac{(r^{(k)}, z^{(k)})}{(Ap^{(k)}, p^{(k)})}, \quad Dz^{(k)} = r^{(k)}. \quad (2.18)$$

Обновление направления выполняется по формуле

$$\beta_k = \frac{(r^{(k)}, z^{(k)})}{(r^{(k-1)}, z^{(k-1)})}, \quad p^{(k)} = z^{(k)} + \beta_k p^{(k-1)}. \quad (2.19)$$

После шага α_k остаток обновляется как

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}. \quad (2.20)$$

Критерий остановки берём, например, по норме остатка:

$$\|r^{(k)}\|_2 \leq \varepsilon \quad \text{или} \quad \frac{\|r^{(k)}\|_2}{\|F\|_2} \leq \varepsilon_{\text{rel}}. \quad (2.21)$$

Задача решаем по следующему алгоритму:

Algorithm 1 Предобусловленный метод сопряжённых градиентов (PCG)

Require: $A, F, u^{(0)}, D, \varepsilon, k_{\max}$

```

1:  $r^{(0)} \leftarrow F - Au^{(0)}$ 
2:  $z^{(0)} \leftarrow D^{-1}r^{(0)}$ 
3:  $p^{(0)} \leftarrow z^{(0)}$ 
4:  $\rho^{(0)} \leftarrow (r^{(0)}, z^{(0)})$ 
5: for  $k = 0, 1, 2, \dots, k_{\max} - 1$  do
6:    $q \leftarrow Ap^{(k)}$ 
7:    $\alpha_k \leftarrow \rho^{(k)} / (p^{(k)}, q)$ 
8:    $u^{(k+1)} \leftarrow u^{(k)} + \alpha_k p^{(k)}$ 
9:    $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k q$ 
10:  if  $\|r^{(k+1)}\|_2 \leq \varepsilon$  then
11:    break
12:  end if
13:   $z^{(k+1)} \leftarrow D^{-1}r^{(k+1)}$ 
14:   $\rho^{(k+1)} \leftarrow (r^{(k+1)}, z^{(k+1)})$ 
15:   $\beta_{k+1} \leftarrow \rho^{(k+1)} / \rho^{(k)}$ 
16:   $p^{(k+1)} \leftarrow z^{(k+1)} + \beta_{k+1}p^{(k)}$ 
17: end for
18: return  $u^{(k+1)}$ 

```

3 Программная реализация и результатов выполнения

3.1 Последовательная

На основе описанных выше методов реализован решатель для уравнения Пуассона с краевыми условиями Дирихле. Решатель построен на использовании класса `PoissonSolver`, определённого в файле `include/conjugate_gradient.hpp`. Данный класс реализует все этапы метода предобусловленного сопряжённого градиента (PCG), включая построение сетки, инициализацию, применение оператора A , предобуславливание и итерационный процесс поиска решения. Метод конечных разностей применяется для аппроксимации оператора Лапласа, а фиктивные области позволяют обрабатывать граничные участки трапециевидной геометрии. Итерации выполняются до достижения нормы остатка, меньшей заданного допуска ε .

Основной файл `task.cpp` организует ввод параметров сетки (M, N) , запуск решателя и измерение времени работы программы. После завершения вычислений решение сохраняется в виде CSV-файла.

При $(M, N) = (10, 10), (20, 20), (40, 40)$ используем последовательную программу для вычисления приближенного решения. Получены следующие результаты:

Сетка ($M \times N$)	Число итераций	Время(s)
(10×10)	20	0.000837
(20×20)	47	0.007440
(40×40)	108	0.069108

Таблица 1: Последовательная программа на разных сеток

3.2 Распараллеливание с OpenMP

В дальнейшем на основе последовательной реализации была добавлена поддержка параллельных вычислений с использованием технологии `OpenMP`. Основная идея заключалась в распараллеливании наиболее трудоёмких двойных циклов по индексам (i, j) , возникающих при вычислении операторов Au , Ap , а также при обновлениях векторов u , r , p и z . Для этого в код были добавлены директивы `#pragma omp parallel for collapse(2)`,

позволяющие автоматически разделять итерации по потокам. Такое решение обеспечивает равномерную загрузку вычислительных ядер и значительно ускоряет процесс решения при больших размерах сетки.

Кроме того, в местах, где использовались операции суммирования (например, при вычислении норм и скалярных произведений), были добавлены конструкции `reduction(+:sum)`, что позволило корректно и безопасно объединять результаты вычислений от разных потоков. Таким образом, все основные участки с независимыми вычислениями были распараллелены, при этом численная точность метода и устойчивость итерационного процесса сохраняются.

При $(M, N) = (40, 40)$ используем последовательную и параллельную программу с 1, 4, 16-нитей. Получены следующие результаты.

Количество нитей	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
-	(40×40)	108	0.069108	1.00
1	(40×40)	108	0.069582	1.00
4	(40×40)	108	0.020581	3.36
16	(40×40)	108	0.012877	5.37

Таблица 2: Распараллеливание с OpenMP с разными количествами нитей

Как показано в таблице 2, при небольших размерах сетки (M, N) объём вычислений остаётся относительно малым, поэтому использование параллельных программ OpenMP не приводит к заметному ускорению по сравнению с последовательной версией. Это связано с тем, что при малом количестве узлов затраты на создание потоков, их синхронизацию и переключение между ними становятся соизмеримыми с самим временем вычислений, что снижает общую эффективность параллелизации.

При увеличении размеров задачи влияние накладных расходов уменьшается, и преимущества параллельных вычислений становятся более очевидными. В дальнейшем мы рассмотрим результаты для более крупных сеток $(M, N) = (400, 600)$ и $(800, 1200)$, где применение OpenMP демонстрирует существенное ускорение по сравнению с последовательной реализацией.

Количество нитей	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
-	(400×600)	1834	10.3237	1.00
2	(400×600)	1834	5.34118	1.93
4	(400×600)	1834	2.77427	3.72
8	(400×600)	1834	1.55052	6.66
16	(400×600)	1812	1.16976	8.83
-	(800×1200)	3759	85.0672	1.00
4	(800×1200)	3759	21.6282	3.93
8	(800×1200)	3759	11.2939	7.53
16	(800×1200)	3759	7.08205	12.01
32	(800×1200)	3759	5.59543	15.20

Таблица 3: Распараллеливание с OpenMP при больших размеров сетки

Анализ представленных данных показывает, что при увеличении размеров сетки эффективность распараллеливания возрастает значительно. Для сетки (400×600) ускорение при 16 потоках достигает значения $S = 8.83$, что близко к линейному масштабированию. При дальнейшем увеличении задачи до (800×1200) достигается ускорение $S = 15.20$ при 32 потоках, что указывает на хорошую масштабируемость метода и эффективное использование вычислительных ресурсов.

Такое поведение объясняется тем, что при больших размерах задачи доля вычислений в общем времени исполнения резко возрастает, а накладные расходы, связанные с управлением потоками, становятся незначительными. Таким образом, реализованный OpenMP-решатель демонстрирует высокую эффективность на крупномасштабных сетках и способен обеспечивать почти идеальную параллельную производительность при увеличении числа вычислительных ядер.

Для наглядного представления полученного приближённого решения уравнения Пуассона наибольшего размера сетки $(M, N) = (800, 1200)$ были построены визуализации в двумерном и трёхмерном форматах. На рисунке 1 показано распределение значений функции $u(x, y)$ на плоскости, что позволяет оценить структуру решения и поведение функции внутри области. Трёхмерная визуализация, представленная на рисунке 2, иллюстрирует рельеф поверхности решения и демонстрирует плавное изменение потенциала по области.

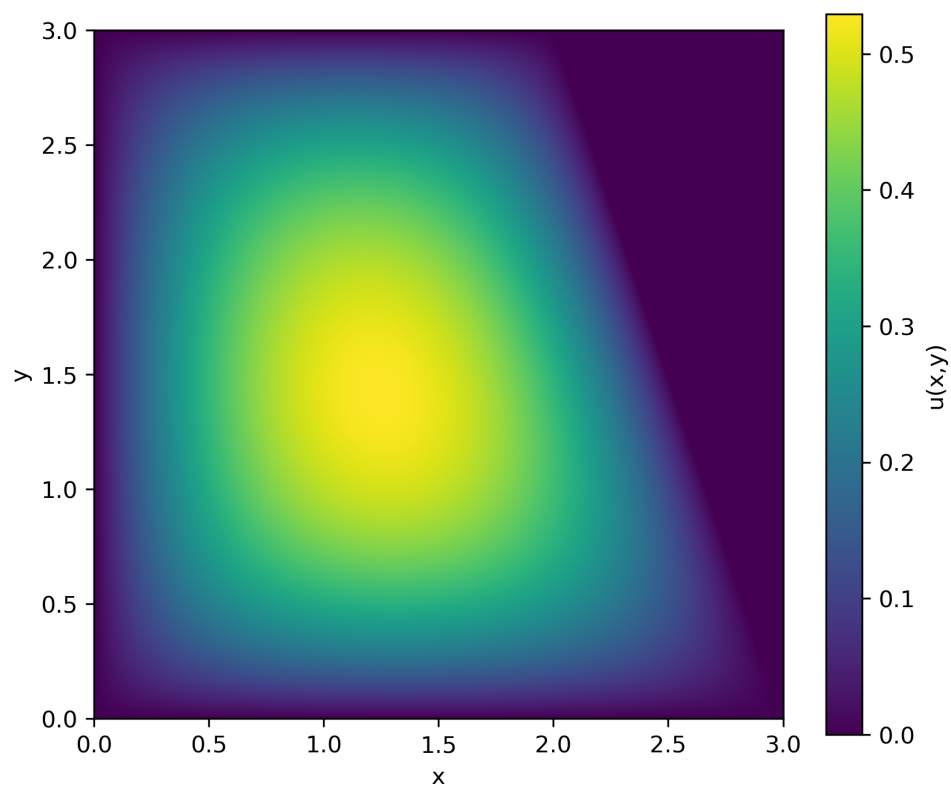


Рис. 1: Двумерная визуализация решения для сетки (800×1200)

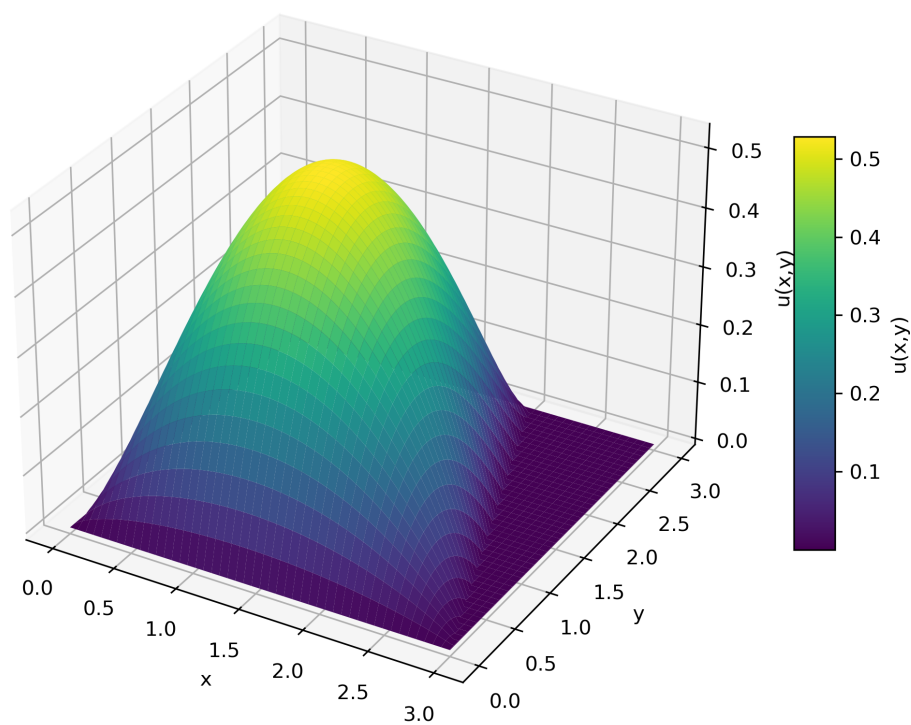


Рис. 2: Трёхмерная визуализация решения для сетки (800×1200)

3.3 Распараллеливание с MPI

Для распределения глобальной прямоугольной сетки по процессам реализован класс `DomainDecomposer`, который строит блочную декомпозицию $p_x \times p_y$ и удовлетворяет двум условиям: (1) отношение сторон каждого локального блока не превышает 2:1, (2) длины подотрезков по каждой оси отличаются не более чем на 1 узел. Для этого выбираются p_x, p_y как близкие к $\sqrt{P}$ множители степени двойки с учётом соотношения $M:N$, а разбиение по каждой оси выполняется как равномерное (блоки размера $\lfloor \frac{\text{size}}{\text{blocks}} \rfloor$ или на единицу больше).

На построенной декомпозиции создаётся декартова топология процессов `MPI_Cart_create` с размерами (p_x, p_y) и без периодичности. Каждому рангу сопоставляются координаты `coords` и его локальный прямоугольник индексов $[x_{\min} : x_{\max}] \times [y_{\min} : y_{\max}]$ (в глобальной нумерации). Далее для каждого процесса конструируется свой `MPIPoissonSolver` на локальной области (с добавлением гало-слоёв по соседям), после чего выполняется итерационный метод.

Ниже показан пример декомпозиции для сетки 800×1200 при $P = 16$:

Пример декомпозиции области для 800×1200 , $P = 16$

```
==== Domain Decomposition Summary ====
Global grid: 800 x 1200
Total processes: 16    Grid layout: 4 x 4
X partitions (4):      0 200 400 600
Y partitions (4):      0 300 600 900
=====
[Rank 0] coords=(0,0) x:[  0, 199] y:[  0, 299]
[Rank 1] coords=(0,1) x:[  0, 199] y:[ 300, 599]
[Rank 2] coords=(0,2) x:[  0, 199] y:[ 600, 899]
[Rank 3] coords=(0,3) x:[  0, 199] y:[ 900,1200]
[Rank 4] coords=(1,0) x:[ 200, 399] y:[  0, 299]
[Rank 5] coords=(1,1) x:[ 200, 399] y:[ 300, 599]
[Rank 6] coords=(1,2) x:[ 200, 399] y:[ 600, 899]
[Rank 7] coords=(1,3) x:[ 200, 399] y:[ 900,1200]
[Rank 8] coords=(2,0) x:[ 400, 599] y:[  0, 299]
[Rank 9] coords=(2,1) x:[ 400, 599] y:[ 300, 599]
[Rank 10] coords=(2,2) x:[ 400, 599] y:[ 600, 899]
[Rank 11] coords=(2,3) x:[ 400, 599] y:[ 900,1200]
[Rank 12] coords=(3,0) x:[ 600, 800] y:[  0, 299]
[Rank 13] coords=(3,1) x:[ 600, 800] y:[ 300, 599]
[Rank 14] coords=(3,2) x:[ 600, 800] y:[ 600, 899]
[Rank 15] coords=(3,3) x:[ 600, 800] y:[ 900,1200]
```

Класс `MPIPoissonSolver` наследует базовый класс `PoissonSolver` и добавляет распределённые операции на основе `MPI`. В конструкторе создаётся декартова топология процессов, а для каждого процесса определяются соседи по четырём направлениям: `left`, `right`, `bottom`, `top`. Это позволяет эффективно организовать обмен граничными слоями (halo exchange) между соседними поддоменами.

Обмен граничными слоями. Для корректного вычисления разностного оператора Лапласа вблизи границ локальной области каждая подзадача хранит дополнительные строки и столбцы «призрачные» узлы. Метод `exchange_halo()` выполняет асинхронный обмен этими данными между соседними процессами с помощью пар вызовов `MPI_Isend/MPI_Irecv` и последующего ожидания `MPI_Waitall`. После завершения обмена граничные значения копируются в соответствующие ячейки сетки, обеспечивая непрерывность решения на стыках поддоменов.

Редукция и вычисление глобальных норм. Поскольку каждая подзадача хранит только часть сетки, скалярные произведения и нормы (например, $\|r\|_2$ или (r, z)) вычисляются локально и затем объединяются во всей топологии с помощью `MPI_Allreduce`.

Завершение и сборка решения. После завершения итерационного процесса каждый процесс хранит свою часть приближённого решения $u(x, y)$. Далее с помощью операции `MPI_Gatherv` данные передаются на `rank 0`, который собирает их в единую глобальную матрицу решения.

Мы также протестировали производительность `MPI` на малых и больших сетках:

Количество процессов	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
(Без <code>MPI</code>)	(40×40)	108	0.069108	1.00
1	(40×40)	108	0.069582	1.00
2	(40×40)	108	0.036846	1.88
4	(40×40)	108	0.020216	3.42

Таблица 4: Распараллеливание с `MPI` с разными количествами процессов

Количество процессов	Сетка ($M \times N$)	Число итераций	Время (s)	Ускорение
1	(400×600)	1834	8.56922	1.00
2	(400×600)	1834	4.55551	1.88
4	(400×600)	1834	2.44147	3.51
8	(400×600)	1834	1.47967	5.79
16	(400×600)	1816	1.24117	6.90
1	(800×1200)	3759	71.3076	1.00
4	(800×1200)	3737	18.0492	3.95
8	(800×1200)	3737	9.61040	7.42
16	(800×1200)	3759	5.71856	12.47
32	(800×1200)	3759	4.91637	14.50

Таблица 5: Распараллеливание с MPI при больших размеров сетки

Анализ экспериментальных результатов показывает, что MPI-параллелизация демонстрирует высокую эффективность и почти линейное ускорение при увеличении числа процессов. Для небольшой сетки (40×40) ускорение ограничено, поскольку объём вычислений невелик и накладные расходы на коммуникацию между процессами сопоставимы с самим временем вычислений. Однако уже при размере (400×600) наблюдается почти идеальная масштабируемость: при 16 процессах ускорение достигает $S = 6.90$.

Для ещё более крупной сетки (800×1200) поведение остаётся аналогичным: при 32 процессах достигается ускорение $S = 14.50$, что подтверждает отличную масштабируемость решателя и эффективность схемы обмена граничными слоями. Незначительные расхождения в числе итераций (в пределах нескольких шагов) связаны с особенностями вычислений в плавающей арифметике: порядок операций при редукции скалярных произведений в MPI может незначительно влиять на накопленную погрешность и, соответственно, на критерий сходимости.

3.4 Распараллеливание с MPI + OpenMP

Для дальнейшего повышения производительности комбинируется распределённая (MPI) и потоковая (OpenMP) параллелизация. В этой гибридной схеме каждая подзадача,

запущенная на своём MPI-процессе, дополнительно использует несколько потоков OpenMP для распараллеливания вложенных циклов по узлам сетки. Такой подход позволяет эффективно задействовать как межузловой, так и внутрипроцессорный параллелизм.

Процессы	Нити	Сетка	Число итераций	Время (s)	Ускорение
-	-	(40×40)	108	0.069108	1.00
1	4	(40×40)	108	0.036172	1.91
2	4	(40×40)	108	0.020501	3.37

Таблица 6: Распараллеливание с MPI+OpenMP с разными количествами процессов и нитей

Процессы	Нити	Сетка	Число итераций	Время (s)	Ускорение
-	-	(400×600)	1834	8.56922	1.00
2	1	(400×600)	1834	5.45576	1.57
2	2	(400×600)	1834	3.77839	2.27
2	4	(400×600)	1834	3.34283	2.56
2	8	(400×600)	1834	3.39928	2.52
-	-	(800×1200)	3759	71.3076	1.00
4	1	(800×1200)	3737	21.4454	3.33
4	2	(800×1200)	3737	14.6345	4.87
4	4	(800×1200)	3756	12.8286	5.56
4	8	(800×1200)	3756	13.0299	5.47

Таблица 7: Распараллеливание с MPI+OpenMP при больших размерах сетки

Как видно из представленных данных, добавление OpenMP-параллелизма поверх MPI обеспечивает дополнительное ускорение, особенно при увеличении числа нитей. Для небольшой сетки (40×40) выигрыш умеренный, однако при увеличении размеров задачи (400×600) и (800×1200) достигается устойчивое ускорение около 2.5 - 5.5 раз при использовании 8 потоков.

3.5 Распараллеливание с MPI + CUDA

Для дальнейшего повышения производительности была реализована версия решателя на основе гибридной модели **MPI + CUDA**. В этой схеме декомпозиция области и коммуникации между процессами выполняются средствами MPI (класс `MPIPoissonSolver`), а наиболее трудоёмкие вычислительные операции переносятся на графические ускорители через класс `MPICudaPoissonSolver`. Реализация организована в виде трёхслойной структуры:

В основе CUDA-реализации лежит трёхуровневая архитектура, включающая набор низкоуровневых CUDA-ядер, систему CUDA-операторов и высокоуровневый класс `MPICudaPoissonSolver`. На первом уровне расположены CUDA-ядра (файлы `cuda_kernels.cu/cuda_kernels.cuh`), выполняющие все вычисления на сетке. К ним относятся ядро `apply_A_kernel`, вычисляющее дискретный дифференциальный оператор $Au = -\nabla \cdot (k\nabla u)$, а также векторные ядра `update_u_kernel`, `update_r_kernel`, `update_p_kernel`, `apply_preconditioner_kernel`, реализующие операции обновления, характерные для метода сопряжённых градиентов. Дополнительно используются ядра `compute_r2_partials_kernel`, `compute_p_Ap_partials_kernel`, `compute_rz_partials_kernel`, которые формируют поэлементные квадраты или произведения векторов для последующей редукции. Все ядра работают на прямоугольной сетке потоков и блоков, используют индексацию `idx(i,j,N+1)` и исключая граничные узлы из вычислений.

Второй уровень реализован в виде набора CUDA-операторов (файл `cuda_operators.cu`), представляющих собой удобные оболочки над ядрами. Они формируют параметры запуска `dim3 grid, block` исходя из локального размера поддомена, принимают CUDA-поток `cudaStream_t stream` для управления асинхронным выполнением, автоматически выполняют проверку ошибок и предоставляют унифицированный интерфейс к операциям. Важной частью является обновлённый механизм редукции, в котором сначала на GPU формируется массив частичных сумм фиксированной длины, а затем выполняется локальная редукция с использованием `thrust::reduce`, что позволяет минимизировать объём данных, пересылаемых с GPU на CPU, и отказаться от попарной редукции в глобальной памяти.

Третий уровень `MPI CUDA Poisson Solver` (файл `mpi_cuda_conjugate_gradient.cpp`), наследующий `MPI Poisson Solver` и интегрирующий вычисления на GPU в MPI-ориентированный итерационный метод. В конструкторе выделяется память на устройстве для всех необходимых массивов (`d_u`, `d_r`, `d_z`, `d_p`, `d_A_p`, `d_k`, `d_M_inv`, `d_partial`), а также создаётся выделенный поток `cudaStream_t`. Методы `to_device` и `to_host` обеспечивают перенос данных между CPU и GPU через промежуточный буфер `h_buf`. Все ключевые функции базового решателя (`compute_l2_norm`, `compute_rz`, `compute_p_Ap`, `update_u`, `update_r`, `update_p`, `apply_preconditioner`, `solve`) переопределяются и реализуются при помощи CUDA-операторов. Отдельно реализован метод `cuda_exchange_halo()`, который копирует граничные слои с GPU на CPU, выполняет обмен через `MPI_Isend/MPI_Irecv` и затем возвращает обновлённые данные обратно на GPU, сохраняя совместимость с существующей MPI-топологией и полностью поддерживая вычисление оператора A на устройстве.

Типы операторов на GPU

В CUDA-версии метода используются три класса вычислительных операторов. Во-первых, полностью параллельные векторные операции, включающие обновление решения, остатка, направления поиска и применение предобуславливателя:

$$u^{(k+1)} = u^{(k)} + \alpha p^{(k)}, \quad r^{(k+1)} = r^{(k)} - \alpha A p^{(k)}, \quad p^{(k+1)} = z^{(k+1)} + \beta p^{(k)}, \quad z = M^{-1}r.$$

Каждая из этих формул реализуется отдельным CUDA-ядром `update_u_kernel`, `update_r_kernel`, `update_p_kernel`, `apply_preconditioner_kernel`. Все операции выполняются независимо для каждого внутреннего узла сетки, что обеспечивает их естественный и полный параллелизм.

Второй тип вычислений дискретный оператор Лапласа, где коэффициенты на полуцелых узлах вычисляются как среднее ближайших значений k . Эта операция реализована в ядре `apply_A_kernel`, вызываемом через интерфейс `cuda_apply_A`. Поскольку оператор использует только локальные соседние значения, вычисления полностью распараллеливаются по узлам сетки и хорошо масштабируются на GPU.

Третий класс операторы редукции, используемые для вычисления норм и скалярных произведений:

$$\|r\|_2^2 = \sum_{i,j} r_{ij}^2, \quad (p, Ap) = \sum_{i,j} p_{ij}(Ap)_{ij}, \quad (r, z) = \sum_{i,j} r_{ij}z_{ij}.$$

Для этих операций используется многоэтапная схема редукции, состоящая из трёх фаз. Сначала GPU формирует массив частичных сумм фиксированной длины при помощи специализированных ядер `compute_r2_partials_kernel`, `compute_pAp_partials_kernel` и `compute_rz_partials_kernel`. Каждое ядро распределяет работу между заранее заданным числом потоков (например, 32768), и каждый поток аккумулирует свой вклад по страйду, записывая одно значение в буфер `d_partial`.

Затем выполняется локальная редукция этого компактного массива непосредственно на устройстве с помощью вызова `thrust::reduce`, который осуществляет суммирование на GPU и возвращает агрегированное значение на хост. Такой подход устраняет необходимость копировать большие массивы и избегает использования разделяемой памяти или warp-специфических примитивов.

После этого на CPU остаётся только выполнить финальную глобальную редукцию по MPI через `MPI_Allreduce`, объединяя локальные скалярные значения всех процессов. Подобная трёхфазная схема минимизирует объём обмена между устройством и хостом и обеспечивает детерминированный и воспроизводимый результат при любых конфигурациях GPU.

Трёхэтапная схема редукции

Редукция норм и скалярных произведений в CUDA-реализации выполняется по трёхэтапной схеме, оптимизированной под архитектуру GPU и ограничения задания.

На первом этапе на GPU формируется массив частичных сумм фиксированной длины. В зависимости от вычисляемой величины запускается одно из специализированных ядер: `compute_r2_partials_kernel`, `compute_pAp_partials_kernel` или `compute_rz_partials_kernel`.

Каждое из этих ядер распределяет работу между фиксированным числом потоков (обычно $128 \times 256 = 32768$), и каждый поток накапливает сумму по страйду, записывая одно

число в массив `d_partial`. Таким образом формируется вектор длины 32768, содержащий агрегированные вклады от всего локального поддомена размером $(M+1)(N+1)$.

На втором этапе выполняется локальная редукция уже компактного массива частичных сумм средствами библиотеки Thrust. Используется вызов: `thrust::reduce()`

Редукция производится целиком на устройстве и возвращает результат на хост, при этом: не используется `__shared__` память отсутствуют warp-специфические примитивы; не происходит копирования больших массивов CPU $\leftrightarrow$ GPU.

Таким образом процесс строго удовлетворяет ограничениям задачи.

На третьем этапе полученное локальное скалярное значение участвует в глобальной редукции MPI.

Этот подход минимизирует объём данных, передаваемых по шине CPU–GPU, разгружает сеть и CPU, и переносит основную вычислительную нагрузку на GPU, что существенно повышает производительность при больших размерах сетки.

Организация обмена граничными слоями на GPU

Для корректной работы оператора Лапласа на границах локальных областей требуется актуальная информация о соседних узлах. В CUDA-версии это реализовано через поэтапную схему обмена гало-слоями между устройством и MPI. Сначала данные граничных слоёв поля `d_p` (или другого массива) копируются с устройства на хост с помощью `cudaMemcpy` в специально выделенные буферы `send_left`, `send_right`, `send_top`, `send_bottom`. Затем на стороне CPU выполняется асинхронный обмен этими буферами с соседними рангами в декартовой топологии с использованием пар вызовов `MPI_Isend/MPI_Irecv`, после чего выполняется коллективное ожидание `MPI_Waitall`. Полученные от соседей данные помещаются в буферы `recv_left`, `recv_right`, `recv_top`, `recv_bottom` и далее копируются обратно на устройство в соответствующие гало-слои массива `d_p`. Такой подход позволяет без изменения логики MPI-версии поддерживать согласованность граничных значений между поддоменами, при этом основные вычисления оператора A и все векторные обновления выполняются целиком на GPU. В данной реализации используется классическая схема обмена через хост, что обеспечивает переносимость и совместимость с используемой MPI-конфигурацией кластера.

Сравнение производительности MPI, MPI+OpenMP и MPI+CUDA

Для оценки эффективности использования графических ускорителей была проведена серия экспериментов на сетках (800×1200) , (1600×2400) и (3200×4800) с одинаковой декомпозицией по MPI-процессам. В таблице ниже приведены результаты:

Процессы	Нити	GPU	Сетка	Время (s)	Итерации	Ускорение
20	-	-	(800×1200)	4.36632	3737	1.00
20	8	-	(800×1200)	3.93555	3737	1.11
1	-	1	(800×1200)	2.66920	3759	1.64
2	-	2	(800×1200)	2.36867	3759	1.84
20	-	-	(1600×2400)	34.9813	7836	1.00
20	8	-	(1600×2400)	34.5548	7836	1.01
1	-	1	(1600×2400)	13.3637	7829	2.62
2	-	2	(1600×2400)	9.34835	7829	3.74
20	-	-	(3200×4800)	328.796	16054	1.00
20	8	-	(3200×4800)	319.480	16054	1.03
1	-	1	(3200×4800)	88.8233	16441	3.70
2	-	2	(3200×4800)	52.8330	16441	6.22

Таблица 8: Результаты MPI, MPI+OpenMP и MPI+CUDA

Как видно, использование технологии CUDA в гибридной реализации **MPI + CUDA** позволяет существенно сократить общее время решения задачи по сравнению с CPU-реализациями. Достижимое ускорение обусловлено переносом наиболее ресурсоёмких вычислительных операций на графический ускоритель, прежде всего вычисления дискретного оператора Лапласа, векторных обновлений метода сопряжённых градиентов, а также локальных этапов редукции.

Применение GPU позволяет эффективно задействовать массовый параллелизм и высокую пропускную способность памяти ускорителя, что особенно важно для регулярных сеточных вычислений и операций типа stencil, характерных для рассматриваемой задачи. При этом вычислительная нагрузка на CPU существенно снижается, а роль центрального

процессора сводится в основном к управлению итерационным процессом и выполнению MPI-коммуникаций.

Следует отметить, что общий выигрыш в производительности формируется за счёт совокупного эффекта ускорения отдельных вычислительных этапов, и его величина зависит от соотношения между вычислениями, операциями редукции и затратами на коммуникации. Поэтому для корректной оценки эффективности GPU-реализации требуется детальный анализ временных затрат по основным категориям.

Точная количественная оценка ускорения, а также подробный разбор распределения времени выполнения по этапам (инициализация, вычисление лапласиана, векторные обновления, редукции, копирование данных и коммуникации MPI) приводятся в следующих разделах отчёта.

3.6 Анализ производительности

В данном разделе проводится анализ производительности решателя уравнения Пуассона для четырёх вариантов параллельной реализации, соответствующих различным вычислительным моделям: распределённой версии на основе MPI с 20 процессами, гибридной версии MPI+OpenMP с 20 MPI-процессами и 8 потоками OpenMP на процесс, а также двух гибридных конфигураций MPI+CUDA с использованием одного и двух графических ускорителей соответственно. Сравнение проводится при фиксированном алгоритме (метод сопряжённых градиентов с предобуславливанием) и одинаковой схеме декомпозиции области, что позволяет корректно оценивать влияние выбранной модели параллелизма и аппаратной архитектуры на итоговую производительность. Для детального рассмотрения структуры временных затрат в качестве показательного примера используется задача на сетке $(M, N) = (3200 \times 4800)$, поскольку при меньших размерах вычислительная нагрузка оказывается недостаточной для полного раскрытия потенциала GPU. Для данной конфигурации анализируется вклад основных групп операторов (дискретного оператора Лапласа, полностью параллельных векторных обновлений и редукций норм и скалярных произведений), а также накладные расходы, связанные с инициализацией, обменом граничными слоями и коллективными операциями MPI; результаты для других размеров сетки и конфигураций параллелизма приведены в приложении отчёта.

MPI-реализация при $P = 20$ для задачи на сетке (3200×4800) демонстрирует типичную для метода сопряжённых градиентов картину распределения вычислительных затрат в условиях распределённой памяти. Основная часть времени приходится на арифметически и алгоритмически доминирующие компоненты: вычисление дискретного оператора Лапласа и векторные обновления, которые в сумме составляют более 65% общего времени работы. Существенную долю занимают также локальные редукции, выполняемые на каждом шаге итерационного метода перед глобальными коллективными операциями. Коммуникационные накладные расходы, связанные с обменом гало-слоями и глобальными редукциями `MPI_Allreduce`, в абсолютных величинах остаются значительно меньше вычислительных этапов, однако в сумме превышают 5% общего времени и выполняются на каждой итерации, что делает их потенциальным ограничивающим фактором при дальнейшем масштабировании по числу процессов. Таким образом, даже при достаточно крупной сетке MPI-реализация остаётся в значительной степени ограниченной затратами на редукции и синхронизацию между процессами.

Этап	Время, с	Доля, %
Инициализация	0.2605	0.08
Оператор Лапласа	101.8440	30.97
Операторы обновления	110.7930	33.69
Локальные редукции	88.7084	26.98
MPI halo exchange	0.7641	0.23
MPI Allreduce	17.2466	5.25
Завершение	8.7546	2.66
Всего	328.7960	100.00

Таблица 9: Профиль временных затрат MPI-реализации (3200×4800) при $P = 20$

MPI+OpenMP-реализация при $P = 20$ и $T = 8$ для задачи на сетке (3200×4800) в целом сохраняет ту же структуру временных затрат, что и чистая MPI-версия, однако демонстрирует частичное перераспределение нагрузки между вычислительными этапами. Использование OpenMP позволяет заметно сократить время локальных редукций за счёт их распараллеливания внутри узла, что приводит к снижению их доли в общем времени

решения. При этом время выполнения оператора Лапласа и векторных обновлений остаётся сопоставимым с MPI-вариантом, а коммуникационные издержки (`MPI halo exchange` и `MPI_Allreduce`) увеличиваются в абсолютных величинах из-за возросшего числа потоков и более частых точек синхронизации. В результате суммарное ускорение по сравнению с MPI-реализацией оказывается умеренным, а общая производительность по-прежнему ограничивается коллективными операциями и обменами между процессами.

Этап	Время, с	Доля, %
Инициализация	0.2649	0.08
Оператор Лапласа	100.4300	31.43
Операторы обновления	116.3190	36.41
Локальные редукции	57.6014	18.02
MPI halo exchange	3.7635	1.18
MPI Allreduce	31.7323	9.93
Завершение	8.7660	2.74
Всего	319.4800	100.00

Таблица 10: Профиль временных затрат MPI+OpenMP-реализации (3200×4800) при $P = 20$, $T = 8$

В гибридной реализации MPI+CUDA для задачи на сетке (3200×4800) при тех же условиях распределения по MPI принципиально меняется профиль временных затрат по сравнению с CPU-ориентированными версиями. Перенос оператора Лапласа, векторных обновлений и локальных редукций на GPU приводит к резкому сокращению общего времени решения: до 88.8 с при использовании одного ускорителя и до 52.8 с при использовании двух GPU. В обоих случаях доминирующими становятся операции, ограниченные пропускной способностью памяти GPU, прежде всего векторные обновления и редукции. При этом вклад коммуникаций MPI и обмена гало-слоями остаётся малым на фоне общего времени, а выигрыш от использования двух GPU проявляется почти пропорционально за счёт сокращения времени CUDA-ядер, что указывает на хорошую масштабируемость вычислительной части при увеличении числа ускорителей.

Этап	Время, с	Доля, %
Инициализация	0.8987	1.01
Оператор Лапласа (CPU)	0.2143	0.24
Оператор Лапласа (CUDA)	13.3782	15.07
Операторы обновления (CPU)	0.2864	0.32
Операторы обновления (CUDA)	44.3312	49.92
Редукции (CUDA)	20.4243	22.99
Копирование Host→Device	0.0974	0.11
Копирование Device→Host	0.0926	0.10
MPI halo exchange	0.0209	0.02
MPI Allreduce	0.0762	0.09
Завершение	9.1579	10.31
Итого	88.8233	100.00

Таблица 11: Профиль временных затрат MPI+CUDA-реализации (3200×4800 , 1 GPU)

Этап	Время, с	Доля, %
Инициализация	0.5784	1.09
Оператор Лапласа (CPU)	0.1072	0.20
Оператор Лапласа (CUDA)	6.8501	12.97
Операторы обновления (CPU)	0.1441	0.27
Операторы обновления (CUDA)	22.4891	42.58
Редукции (CUDA)	11.2421	21.28
Копирование Host→Device	0.5712	1.08
Копирование Device→Host	1.1483	2.17
MPI halo exchange	0.1642	0.31
MPI Allreduce	0.2670	0.51
Завершение	9.1423	17.30
Итого	52.8330	100.00

Таблица 12: Профиль временных затрат MPI+CUDA-реализации (3200×4800 , 2 GPU)

4 Заключение

В данной работе был разработан и всесторонне исследован вычислительный комплекс для решения уравнения Пуассона в трапециевидной области методом предобусловленных сопряжённых градиентов (PCG). Ключевой особенностью проекта является последовательная эволюция вычислительной модели — от базовой последовательной реализации к многоуровневым параллельным схемам OpenMP, MPI и гибридной архитектуре MPI+CUDA. Такой подход позволил не только добиться существенного ускорения вычислений, но и систематически проанализировать влияние различных форм параллелизма на структуру временных затрат и масштабируемость алгоритма.

Разработанная программная архитектура отличается высокой модульностью и расширяемостью. Использование класса `DomainDecomposer` обеспечивает автоматическое построение сбалансированных MPI-декомпозиций области с сохранением геометрических пропорций поддоменов, что является важным условием устойчивой масштабируемости. Поддержка гибридной модели MPI+CUDA реализована через класс `MPIcudaPoissonSolver`, который органично расширяет базовую MPI-логику, перенося наиболее трудоёмкие вычислительные этапы на графический ускоритель. Принятая трёхуровневая архитектура CUDA («ядра $\rightarrow$ операторы $\rightarrow$ высокоуровневый алгоритм») обеспечивает ясное разделение ответственности, упрощает отладку и делает код пригодным для дальнейшего развития.

Подробный анализ производительности выявил характерные особенности каждой из реализаций. В последовательной версии основную долю времени ожидаемо занимают оператор Лапласа и операции редукции. Параллелизация с использованием OpenMP позволяет существенно сократить время выполнения вычислительных ядер на одном узле, однако эффективность ограничивается пропускной способностью памяти и числом доступных ядер. MPI-реализация демонстрирует хорошую вычислительную эффективность при умеренном числе процессов, при этом вклад коммуникаций остаётся относительно малым, но наличие глобальных редукций накладывает фундаментальные ограничения на масштабируемость при увеличении P .

Гибридная реализация MPI+CUDA показала наилучшие результаты. Перенос оператора Лапласа, векторных обновлений и части редукций на GPU приводит к резкому сокращению общего времени решения при сохранении той же численной схемы PCG.

При увеличении размера сетки достигается устойчивый рост ускорения, что согласуется с теоретическими оценками, основанными на соотношении вычислительной мощности и пропускной способности памяти CPU и GPU. При этом профиль временных затрат смещается от вычислений оператора Лапласа к операциям редукции и обменов, что отражает переход к конфигурации, ограниченной пропускной способностью памяти и синхронизациями. Тем не менее даже в этих условиях гибридная версия обеспечивает ускорение на порядок и более по сравнению с CPU-реализациями, что подтверждает эффективность выбранного подхода.

В результате выполненной работы создан законченный и надёжный программный комплекс, включающий реализации для CPU, OpenMP, MPI, MPI+OpenMP и MPI+CUDA, а также средства автоматизированного запуска и детального профилирования на суперкомпьютерной системе. Полученные результаты наглядно демонстрируют, что при сохранении одного и того же численного алгоритма увеличение объёма вычислений и перенос их на современные графические ускорители позволяет в полной мере раскрыть потенциал архитектур GPU. Представленный комплекс может служить как учебным примером построения гибридных HPC-приложений, так и основой для дальнейших исследований и практических инженерных расчётов в области численного моделирования и высокопроизводительных вычислений.